Predicting Coronary Heart Disease

GENILLOUD Erine, COLOMBIES Lou, ACHTARI Chloé
CS433-Machine Learning, EPFL

***Abstract*—We predict coronary heart disease (MICHD) from the answers of 328'135 participants to a Behavioral Risk Factor Surveillance System (BRFSS) telephone survey. Our code cleans the raw survey answers, adds pairwise interactions between cardiovascular risk factors, and trains an L2-regularized logistic regression with Newton's method. Because only 8.8 % of the participants are positive, the decision threshold is tuned for the F1 score by stratified 5-fold cross-validation. Our final model reaches a cross-validated F1 of 0.432 ± 0.005. We found that tuning the threshold has the largest impact, while feature engineering brings a small but consistent improvement.**

I. Introduction

Cardiovascular diseases are one of the leading causes of death worldwide, and early identification of people at risk can help prevention. In the BRFSS 2015 survey, participants are labelled positive for MICHD if they have been told by a health professional that they had coronary heart disease, a heart attack or angina. The training set contains $N = 328'135$ participants with 321 raw features, and the test set $109'379$ participants. Since the classes are imbalanced ($8.8\,\%$ positives), we evaluate our models with the F1 score of the positive class rather than with accuracy: predicting "healthy" for everyone already gives $91\,\%$ accuracy but an F1 of 0.

II. Data preprocessing

All steps are in `preprocessing.py`. To avoid any information leak, every statistic (medians, clipping bounds, means and standard deviations) is computed on the training part of each fold only, and then applied to the validation part.

Feature selection. Using the BRFSS codebook, we kept 65 variables related to health status, chronic diseases, demographics, body mass index, smoking, alcohol, diet, physical activity and access to care. We removed administrative columns, sampling weights and phone-call logistics, as well as `HAREHAB1` (cardiac rehabilitation), which is only asked to people who already had a heart attack and therefore contains the answer.

Cleaning. Answers "don't know" or "refused" (codes 7/9, 77/99) were set to missing, the code "none" (88) to 0, and yes/no answers recoded to $1/0$. Some questions are only asked depending on a previous answer; in that case the missing value has a known meaning and we filled it with its logical value (e.g. 0 minutes of activity for people who do not exercise).

Missing values and scaling. We dropped variables with more than $40\,\%$ missing values. For numerical variables, we replaced extreme values by the 1st and 99th percentiles, and took the logarithm of very skewed ones (alcohol, minutes of activity). We filled missing values with the median, or with the most frequent value for yes/no variables, and added a column saying whether the value was missing when a variable had more than $5\,\%$ missing values. Variables with categories but no order (e.g. marital status) were one-hot encoded, with "missing" as an extra category. Finally, we standardized the numerical variables. This gives 119 features (our baseline).

Feature engineering. Logistic regression is linear in its features, so non-linear effects must be given explicitly. We added (i) one-hot encodings of age group, BMI category and general health, whose effect on the risk is not linear (the positive rate goes from $0.5\,\%$ at 18–24 years to $22\,\%$ above 80), (ii) the squares of age and BMI, (iii) four questions that were only asked in some states (daily aspirin, insulin, prediabetes, memory loss); with 70–99 % missing values, we one-hot encoded them with "no answer" as a separate category instead of filling them, and (iv) all 105 pair products between 15 major risk factors (e.g. age, sex, diabetes, high blood pressure, high cholesterol, smoking, BMI). This gives 264 features.

III. Models and methods

Implemented methods. We implemented the six required methods (`implementations.py`): linear regression by gradient descent, stochastic gradient descent and normal equations, ridge regression, and logistic regression without and with L2 regularization by gradient descent.

Explored models. We also tried k-nearest neighbors (KNN), but with large n and large d, kNN is impractical, as each prediction requires computing distances to all training points in a high-dimensional space, which is both costly in time and memory and less meaningful due to the curse of dimensionality. We also implemented a linear SVM with class weights to compensate for the imbalance. On the same validation set and with a tuned threshold, the SVM reached an F1 of 0.420, below logistic regression (0.429). It was also much slower: after 10 passes over the data it had not fully converged (duality gap of 0.03), and more than half of the training points were support vectors. Since logistic regression was better, faster and gives probabilities, we kept it.

Final model. We minimize the regularized logistic loss

$$L(w) = \frac{1}{N} \sum_{n=1}^{N} \left[\log\left(1 + e^{x_n^\top w}\right) - y_n x_n^\top w \right] + \lambda \|w\|^2, \quad (1)$$

where the bias is not penalized, with Newton's method, $w \leftarrow w - H^{-1} \nabla L(w)$, where $H = \frac{1}{N} X^\top S X + 2\lambda I$ and $S = \mathrm{diag}(\sigma(x_n^\top w)(1 - \sigma(x_n^\top w)))$. The objective is convex, so Newton's method reaches the same solution as gradient descent, but in about 10 iterations instead of hundreds, without tuning a step size. This made the cross-validation of many configurations fast.

Decision threshold. Logistic regression gives for each participant a probability of being sick, $\sigma(x^\top w)$, and by default we predict "sick" when it is above 0.5. Since only $8.8\,\%$ of the participants are positive, the model rarely gives such a probability: with this threshold it detects only $14\,\%$ of the sick participants. We therefore predict positive when $x^\top w > t$, and choose the threshold t that maximizes F1. Lowering the threshold detects more sick participants but also creates more false alarms. Our best threshold, $t = -1.36$, corresponds to a probability of about $20\,\%$, more than twice the average risk.

Cross-validation. We use stratified 5-fold cross-validation: the training set is split into 5 parts with the same proportion of sick participants. Preprocessing and model are re-fitted on each training fold, so that every participant gets a prediction score from a model that never saw them. We choose the threshold on these scores, compute the F1 on each fold and report its mean and standard deviation. The final model is retrained on the full training set and applied to the test set with this threshold.

Ablation study. To measure how much each modification actually helps, we start from the baseline and add them one at a time, measuring the F1 with the same 5 folds after each step. We can then count on how many folds a modification improves the F1: one that improves only the mean, but not consistently, is likely just noise. We chose λ the same way, testing values from 10^{-6} to 10^{-2}.

IV. Results

Table I compares the models with the baseline preprocessing. Without threshold tuning, logistic regression predicts almost everyone as healthy: its accuracy is high but its F1 is low. Tuning the threshold was the most important step (F1 from 0.22 to 0.42). Logistic regression then outperforms ridge regression, whose squared loss is not adapted to classification, and gradient descent, which has not yet converged after 300 iterations.

TABLE I
Model comparison (5-fold CV, $\lambda = 10^{-4}$).

Model	F1	Accuracy
Trivial (all positive)	0.162	0.088
Ridge regression (± 1 labels), tuned t	0.410 ± 0.006	0.857
Reg. logistic, GD (300 it.), tuned t	0.417 ± 0.003	0.867
Reg. logistic, Newton, t = 0.5	0.223 ± 0.006	0.914
Reg. logistic, Newton, tuned t	0.424 ± 0.006	0.871
+ feature engineering (final)	$\mathbf{0.432 \pm 0.005}$	0.871

In the ablation study (Table II), the one-hot encodings and squared terms alone do not change the F1. The optional modules improve it on 4 of the 5 folds, and above all the 105 risk-factor interactions improve it on all five folds. Adding the state of residence (54 one-hot columns) does not help, so we did not keep it. F1 is stable for $\lambda \leq 10^{-4}$ (0.431) and decreases for larger values (0.429 at 10^{-3}, 0.423 at 10^{-2}); we kept $\lambda = 10^{-4}$.

Our final model predicts $13.9\,\%$ of the test participants as positive.

TABLE II
Ablation of the feature engineering (D: number of features; Newton, tuned threshold, 5-fold CV). Impr.: number of folds where F1 is higher than the baseline.

Configuration (cumulative)	D	F1	Impr.
Baseline preprocessing	119	0.4241 ± 0.0058	–
+ one-hots, age^2, BMI^2	146	0.4243 ± 0.0058	2/5
+ 12 targeted interactions	158	0.4257 ± 0.0052	4/5
+ optional modules	171	0.4268 ± 0.0053	4/5
+ 105 pairwise inter. (final)	264	$\mathbf{0.4315 \pm 0.0048}$	5/5
+ state (one-hot)	318	0.4312 ± 0.0048	5/5

V. Discussion

The gain from the interactions is small ($+0.007$) but consistent across folds, which suggests that risk factors combine (e.g. diabetes matters more for older people), something a linear model cannot capture by itself. The F1 remains moderate (≈ 0.43): the data are self-reported, and many people with risk factors never develop the disease.

Limitations. Some features may be consequences rather than causes of the disease: for example daily aspirin is often prescribed after a cardiac event. Such features improve prediction, but the model should not be interpreted causally. Our final model is also linear; other non-linear models could capture more interactions.